



PROYECTO FINAL

Sistema ETL paralelo para el procesamiento y consolidación de ventas de múltiples sucursales

DATOS

Universidad: Instituto Tecnológico de Las Américas (ITLA)

Materia: Programacion Paralela

Docente: Erick Leonardo Perez

Sección: Lunes de 8:00 pm - 10:00pm y Miercoles de 8 pm a - 10 pm

Datos De los Estudiantes

- Ranniel Muñoz 2024-2567
- Kalil Francisco Camilo 2023-1153
- Mario De Jesus Suero 2024-0263

Fecha de entrega: 19/08/2026

Métricas del sistema de ventas ETL.

En este documento estarán los datos obtenidos en cuatro ejecuciones experimentales con una carga masiva de 50 millones de registros de transacciones en una computadora con cuatro procesadores lógicos.

Primer análisis de métricas.

```
=====
SISTEMA ETL PARALELO DE VENTAS MULTISUCURSAL (C# / .NET)
=====
Procesadores lógicos disponibles en el equipo: 4

Ingrese la cantidad de procesadores a utilizar: 1
Ingrese la cantidad de registros masivos a simular (ej. 10000000): 50000000

[1/3] Generando transacciones masivas en memoria...
[2/3] Ejecutando procesamiento ETL (Secuencial vs Paralelo)...

=====
RESULTADOS Y TELEMETRÍA
=====
Tiempo Secuencial:                711.32 ms
-----
MÉTODO 1: Descomposición de Datos (Bloques)
Tiempo Paralelo:                  847.10 ms
Speedup (Aceleración):            0.84x
Eficiencia:                       83.97%
¿Resultados coinciden?:           True
-----
MÉTODO 2: Optimización con Datos Locales (Parallel.For) [Recomendado]
Tiempo Paralelo:                  1303.33 ms
Speedup (Aceleración):            0.55x
Eficiencia:                       54.58%
¿Resultados coinciden?:           True
=====

Proceso finalizado. Presione cualquier tecla para salir...

C:\Users\Kalil Camilo\Documents\GitHub\Proyecto-final-de-programaci-n-paralela\src\Sistema ETL de de ventas\Sistema ETL
de de ventas\bin\Debug\Sistema ETL de de ventas.exe (proceso 13668) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->
Cerrar la consola automáticamente al detenerse la depuración.
```

En este primer análisis se puede demostrar que al forzar la ejecución a utilizar un solo hilo o núcleo, los métodos paralelos pueden resultar más lentos que el enfoque secuencial tradicional. Esto se puede ver a la sobrecarga de introducción por la inicialización de las tareas, en donde la partición y los delegados de la librería de paralelismo sin disponer de núcleos adicionales para compensar el costo en el rendimiento.

Segundo análisis de métricas

```
Ingrese la cantidad de registros masivos a simular (ej. 10000000): 50000000

[1/3] Generando transacciones masivas en memoria...
[2/3] Ejecutando procesamiento ETL (Secuencial vs Paralelo)...

=====
RESULTADOS Y TELEMETRÍA
=====
Tiempo Secuencial:                692.69 ms
-----
MÉTODO 1: Descomposición de Datos (Bloques)
Tiempo Paralelo:                  487.82 ms
Speedup (Aceleración):            1.42x
Eficiencia:                       71.00%
¿Resultados coinciden?:           True
-----
MÉTODO 2: Optimización con Datos Locales (Parallel.For) [Recomendado]
Tiempo Paralelo:                  797.58 ms
Speedup (Aceleración):            0.87x
Eficiencia:                       43.42%
¿Resultados coinciden?:           True
=====

Proceso finalizado. Presione cualquier tecla para salir...

C:\Users\Kalil Camilo\Documents\GitHub\Proyecto-final-de-programaci-n-paralela\src\Sistema ETL de de ventas\Sistema ETL
de de ventas\bin\Debug\Sistema ETL de de ventas.exe (proceso 21800) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->
Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```

En este segundo análisis se puede notar que con dos núcleos de procesamiento, el método uno logra una aceleración bastante clara de 1.42x, reduciendo el tiempo de procesamiento a 487.82 ms con un aprovechamiento del hardware del 71%. Y también el segundo método reduce significativamente el tiempo respecto al primer caso con un solo núcleo de procesamiento, aunque a un reciente reciente costo de abstracción con el parallel.for para esta carga en particular.

Tercer análisis de métricas

```
=====
SISTEMA ETL PARALELO DE VENTAS MULTISUCURSAL (C# / .NET)
=====
Procesadores lógicos disponibles en el equipo: 4

Ingrese la cantidad de procesadores a utilizar: 3
Ingrese la cantidad de registros masivos a simular (ej. 10000000): 50000000

[1/3] Generando transacciones masivas en memoria...
[2/3] Ejecutando procesamiento ETL (Secuencial vs Paralelo)...

=====
RESULTADOS Y TELEMETRÍA
=====
Tiempo Secuencial: 788.05 ms
-----
MÉTODO 1: Descomposición de Datos (Bloques)
Tiempo Paralelo: 477.25 ms
Speedup (Aceleración): 1.65x
Eficiencia: 55.04%
¿Resultados coinciden?: True
-----
MÉTODO 2: Optimización con Datos Locales (Parallel.For) [Recomendado]
Tiempo Paralelo: 642.79 ms
Speedup (Aceleración): 1.23x
Eficiencia: 40.87%
¿Resultados coinciden?: True
=====
```

Para este segundo análisis de las métricas, se puede notar que ambos algoritmos superan la velocidad secuencial. El método 1 llega a escalar a 1.65x, mientras que el segundo método cruza el umbral de ganancia con 1.523x, en donde la eficiencia porcentual comienza a descender por el contenido de los buses de memoria al procesar 50 millones de registros simultáneamente.

Cuarto análisis de métricas

```
=====
SISTEMA ETL PARALELO DE VENTAS MULTISUCURSAL (C# / .NET)
=====
Procesadores lógicos disponibles en el equipo: 4

Ingrese la cantidad de procesadores a utilizar: 4
Ingrese la cantidad de registros masivos a simular (ej. 10000000): 50000000

[1/3] Generando transacciones masivas en memoria...
[2/3] Ejecutando procesamiento ETL (Secuencial vs Paralelo)...

=====
RESULTADOS Y TELEMETRÍA
=====
Tiempo Secuencial: 708.38 ms
-----
MÉTODO 1: Descomposición de Datos (Bloques)
Tiempo Paralelo: 385.09 ms
Speedup (Aceleración): 1.84x
Eficiencia: 45.99%
¿Resultados coinciden?: True
-----
MÉTODO 2: Optimización con Datos Locales (Parallel.For) [Recomendado]
Tiempo Paralelo: 535.55 ms
Speedup (Aceleración): 1.32x
Eficiencia: 33.07%
¿Resultados coinciden?: True
=====

Proceso finalizado. Presione cualquier tecla para salir...

C:\Users\Kalil Camilo\Documents\GitHub\Proyecto-final-de-programaci-n-paralela\src\Sistema ETL de de ventas\Sistema ETL
```

Ya para el último y el cuarto análisis, aquí se ve que es donde se alcanza mejor tiempo global en el primer método con 385.09 ms, casi el doble de rápido que la versión secuencial con un speedup de 1.84x, y el método 2 logra su mejor marca con 535.55 m. Aquí se puede validar la integridad en donde todas las ejecuciones son true y confirman que no extendieron las coincidencias de carrera durante las agregaciones en paralelo.